Errors Are Not Choke Points: Difficulty, Error Onset and Counterfactual Commitment in LLM Reasoning Failures

Harsh Dubey Harshit Saini Prashant Jain
Himanshu Kumar Gaurav Kaushal Rohit Chaudhary
Dept. of CSE (AI&ML), KIET Group of Institutions, Ghaziabad, India
harsh.2428cseaiml127@kiet.edu

Abstract

Process supervision assumes that the first incorrect step of a reasoning trace is where it fails; verifier-guided search backtracks to that step, and early-warning detectors assume failure is visible in a partial trace. We test these assumptions on 3,400 ProcessBench solutions with expert first-error labels. A detector that reads only the question and the generator's identity predicts a wrong answer with AUROC 0.588; a TF-IDF detector reading the first quarter of the reasoning adds nothing beyond it (+0.005, n.s.) and is at chance within a problem. 23.5% of annotated first errors are followed by a correct answer, fatality does not depend on where the error occurs, and failure detectors rank these non-fatal errors as riskier than fatal ones (AUROC 0.456). We then measure commitment counterfactually, scoring 50,152 continuations sampled from three open models at every prefix of their own traces. 33–66% of failures are decided before the first step (the model almost never solves the problem). When the outcome is still open, the annotated error is the largest drop in success probability in 0.344–0.529 of traces (position-matched null 0.159–0.214). Most strikingly, resampling from just before the expert-labelled first error succeeds less often than restarting from scratch in all three models (by +0.068 to +0.215), whereas resampling from where success actually drops matches restarting with 15–29% fewer tokens. Verifiers that steer search should target drops in success probability, not first errors.

1 Introduction

Chain-of-thought (CoT) reasoning [1] turns a single prediction into a trajectory of steps, and two widely used ideas assume that a wrong final answer is decided at an identifiable point inside that trajectory. Process supervision [2, 3, 5] trains and evaluates verifiers to find the *first incorrect step*, and verifier-guided search backtracks to it. Early-warning detectors [9, 10] predict from a partial trajectory that the answer will be wrong, so that computation can be stopped or redirected.

Both ideas rest on two assumptions that are rarely tested. The first is that the signal they use is about the *trajectory*: a detector that reads a prefix should know more than a detector that reads only the question. The second is that the *annotated first error is the point at which the outcome is decided*. We call a step that decides the outcome a *choke point*, and ask whether errors and choke points coincide.

We test both assumptions on ProcessBench [5], which provides expert first-error labels for 3,400 solutions written by 12 open models, and we contribute:

1. **A difficulty-controlled protocol for early failure detection.** A detector that reads only the question and the generator's identity reaches AUROC 0.588; reading the first quarter of the reasoning on top of that adds +0.005 (n.s.), and within a problem the 25% prefix is at chance (0.485). Reasoning text adds +0.022–+0.031 AUROC only after half the chain.
2. **Errors are not choke points, observationally.** 23.5% of first errors are followed by a correct answer (34.1% on Omni-MATH), fatality does not depend on where the error occurs, and a coded sample shows these are rarely repairs.
3. **Failure detectors track error presence, not commitment.** They rank non-fatal errors as riskier than fatal ones (AUROC 0.456), and once an error has occurred its fatality is not predictable from the text.
4. **A counterfactual measurement of commitment.** For traces written by three open models we resample the *same* model from every prefix (50,152 scored continuations) and estimate the success probability v_k after each step. 33–66% of failures are capability-limited before the first step; when the outcome is still open, the annotated error is where success collapses in 0.344–0.529 of traces, against a position-matched null of 0.159–0.214 (Section 5).
5. **First-error backtracking loses to restarting.** With sample splitting, resampling from just before the expert-labelled first error succeeds less often than restarting from scratch in all three models (by +0.068 to +0.215), while resampling from just before the largest drop in v matches restarting with 15–29% fewer tokens. A simple super-martingale argument explains why restart is a strong baseline for any backtracking target that is not chosen with information about where success is lost.

All code, tables and the generator of every number in this paper are released.[1]

[1] https://github.com/HarshDubey23/early-failure-detection-cot

2 Related Work

Process supervision. Step-level supervision outperforms outcome supervision [2, 3]; Monte Carlo rollouts can label steps automatically [4], but such labels disagree with human judgements because a completion can reach the right answer from a wrong step and vice versa, which Zhang et al. [6] treat as label noise. ProcessBench [5] evaluates first-error identification. We study the disagreement itself.

Predicting failure early. Models carry information about their own correctness [22, 21], and hidden states predict CoT success before or during generation [9, 10]; conformal methods certify reasoning prefixes [20]. Bulut [11] shows that such pooled AUROCs are largely explained by problem difficulty and fall to chance within a problem. We apply the same control to text detectors on human-annotated traces.

Which steps matter. Resampling from prefixes identifies influential sentences [12], tokens [13, 14] and the importance of the first step [15]. Merrill and Srivastava [16] localise the point of no return for deceptive behaviour and name incorrect solutions as future work; recoverability of error states has been studied for distillation [17]. None compares a counterfactual decision point with human first-error labels. Work on performative CoT [18, 19] finds that answers are often fixed apart from the written reasoning, which our non-fatal errors echo.

Correcting and backtracking. Unaided self-correction rarely helps [8], but models can correct a mistake once told where it is [7], which motivates verifier-guided search that returns to the first error. Independent resampling from scratch, as in self-consistency [23], is the natural baseline for any such scheme; we compare the two directly.

3 Data and Definitions

Data. ProcessBench [5] contains 3,400 step-segmented solutions to GSM8K, MATH, OlympiadBench and Omni-MATH problems (2,842 distinct problems) from 12 open models, each with the index e of the first erroneous step (or none) and whether the final answer is correct. Correct and incorrect answers are balanced within each tier by construction.

Outcome groups. A trace is *clean* (no annotated error; 1,179), *recovered* (an error, but a correct answer; 521) or *fatal* (an error and a wrong answer; 1,700). No clean trace has a wrong answer. The *fatality* of first errors is $P(\text{wrong} \mid \text{error})$.

Counterfactual value. For a trace $T = (s_1, \ldots, s_n)$ written by model M, let $v_k = P_M(\text{correct} \mid q, s_{1:k})$, estimated by sampling m continuations of the prefix from M; v_0 is M's from-scratch solve rate and v_n is the observed outcome. The *decisive step* is $d = \arg\max_k(v_{k-1} - v_k)$, and the *point of no return* is $c = \min\{k : v_j \leq \delta \ \forall j \geq k\}$. A failing trace is *capability-limited* if $v_0 \leq \delta$, shows *latent commitment* if $c < e$, *commits at the error* if $c = e$, and has a *recoverable error* if $c > e$.

Statistics. Detectors are TF-IDF (uni+bigrams) or one-hot features with logistic regression [28], cross-fitted over 5 folds grouped by problem and repeated over 5 seeds. Intervals are 95% question-level cluster bootstrap intervals [27] (1,000 resamples); differences between detectors use a paired bootstrap on the same traces.

Table 1: AUROC for a wrong final answer (out-of-fold, 3,400 traces). Q: question text; G: generator and tier.

Detector	AUROC	95% CI	SD (5 seeds)
Q	0.561	0.543–0.578	0.005
G	0.539	0.521–0.557	0.002
Q+G	0.588	0.570–0.607	0.003
Prefix 25%	0.564	0.545–0.582	0.006
Prefix 100%	0.600	0.582–0.619	0.004
Q+G+prefix 25%	0.593	0.575–0.611	0.004
Q+G+prefix 50%	0.610	0.592–0.628	0.004
Q+G+prefix 100%	0.619	0.601–0.637	0.003
Trace length	0.542	0.524–0.561	0.002

4 Observational Results

4.1 Early failure prediction is a difficulty signal

Table 1 and Fig. 1 compare detectors that read the question, the generator and tier, the reasoning prefix, or combinations. The question alone reaches 0.561 and question plus generator 0.588 [0.570, 0.607]. A detector reading only the first 25% of the reasoning is *worse* than that (-0.025, $p = 0.012$), and adding the 25% prefix to question and generator changes AUROC by +0.005 [-0.004, +0.014]. The reasoning adds +0.022 at 50% and +0.031 [+0.020, +0.041] at 100%, consistently across all five seeds.

Within a problem the generator is the only varying factor besides the reasoning: over 326 problems with both a failing and a succeeding trace (466 pairs), the 25% prefix orders pairs correctly 0.485 of the time [0.440, 0.536] and the full prefix 0.539 [0.487, 0.589], neither better than generator identity alone (0.548). Early failure prediction on this benchmark is therefore almost entirely a statement about which problems are hard for which models.

4.2 One first error in four is not fatal

Fatality of first errors is 0.765 [0.748, 0.782]: 23.5% of annotated first errors are followed by a correct answer (Fig. 2). Fatality falls with difficulty, from 0.966 on GSM8K to 0.659 on Omni-MATH, and is flat in the position of the error (0.749–0.766 across onset quartiles; 0.890 only when the error is in the final step). Fatal and recovered errors occur at the same relative position (Mann–Whitney $p = 0.21$, $r = 0.04$).

We read and coded a seeded sample of 40 recovered traces. In 24 the reasoning stays wrong and the correct answer appears anyway; 6 have low-entropy or qualitative answers (yes/no, true/false); in 5 the error does not bear on the answer; only 4 contain a repair; 1 flagged step looked correct to the coder. Explicit re-checking language after the error is barely more frequent in recovered than in fatal traces (18.6% vs 15.5%,

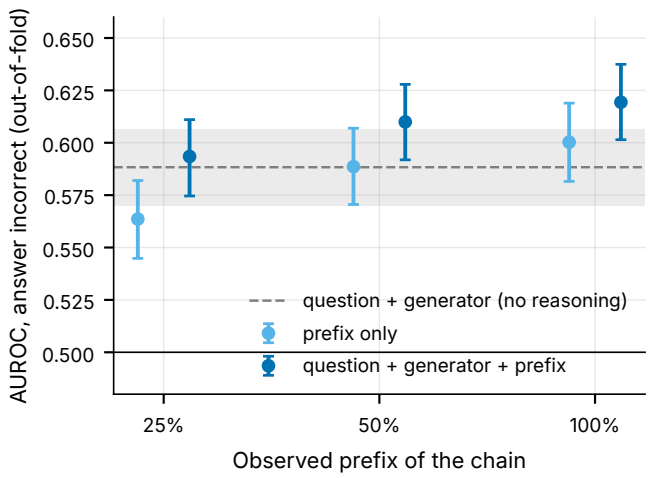


Figure 1: The prefix adds nothing beyond question and generator until half the chain is observed.

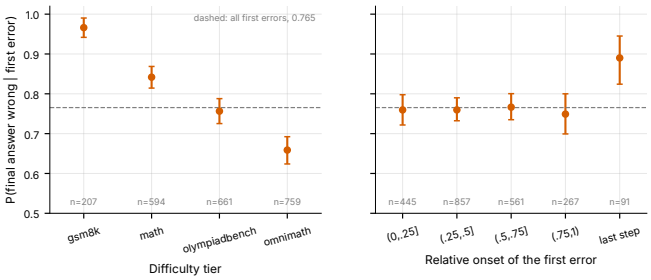


Figure 2: Fatality of first errors by difficulty tier and by where the error occurs.

$p = 0.09$). Non-fatal errors are thus mostly cases where the written reasoning does not determine the answer, consistent with [6, 18].

4.3 Detectors track errors, not commitment

If a failure detector captured commitment, it would score recovered traces like clean ones. It does the opposite (Table 2): trained to predict a wrong answer, the full-prefix detector separates recovered from clean traces (0.703) but ranks fatal traces *below* recovered ones (0.456 [0.428, 0.484]); with question and generator included the ranking is at chance (0.501). A detector trained to find errors inverts more strongly (0.376). Among erroneous traces, the fatality of the first error is predicted by question and generator (AUROC 0.671), and the error step, the prefix up to it or the next one or two steps change this by at most 0.003.

5 Counterfactual Commitment

The observational results say that first errors and outcomes come apart, but not where a failing trajectory actually becomes unrecoverable. We measure that directly by resampling.

Table 2: AUROC between outcome groups (positive vs negative).

Detector (trained on)	rec. vs clean	fatal vs rec.	fatal vs clean
Prefix 100% (wrong answer)	0.703	0.456	0.664
Q+G+prefix (wrong answer)	0.679	0.501	0.672
Prefix 100% (has error)	0.819	0.376	0.709

5.1 Setup

Generators and traces. We use the three ProcessBench generators whose weights fit on two 16 GB GPUs: Qwen2.5-1.5B-Instruct and Qwen2-1.5B-Instruct [24], all of whose traces we use (116 and 88), and Qwen2.5-Math-7B-Instruct [25], for which we use all fatal and recovered traces with a recoverable gold answer and 60 clean traces (314 in total; 3 traces without a gold answer are dropped). Gold answers come from the source datasets, matched on problem text, and answers are compared with `math-verify`.

Rollouts. For every prefix $s_{1:k}$, $k = 0, \ldots, n-1$, we sample m continuations from the trace's own generator (16, 16 and 12 respectively; temperature 0.7, top-p 0.95; at most 1,536 new tokens for the 1.5B models and 1,024 for the 7B model) with vLLM [26] in float16, and estimate v_k as the fraction that reach the gold answer. In total we score 50,152 continuations from 704, 446 and 2,646 prefixes. Every continuation, its answer and its correctness are released.

What can and cannot be read off v_k. Under the generating policy, v_k is a martingale. Curves conditioned on the eventual outcome therefore drift even when no single step matters: for any fixed k (and any stopping time),

$$\mathbb{E}[v_k \mid \text{fail}] = \frac{\mathbb{E}[v_k(1-v_k)]}{P(\text{fail})} \leq v_0, \qquad (1)$$

because $x(1-x)$ is concave, so $v_k(1-v_k)$ is a supermartingale. We therefore never interpret a downward drift of failing curves by itself. We ask instead whether the drop is *concentrated* at the annotated error, against a position-matched null and the drop noise floor of clean traces, and we evaluate backtracking policies with sample splitting.

5.2 Where failing trajectories commit

Table 3 classifies each fatal trace at $\delta = 1/8$.

Many failures are decided before the first step. 33%, 52% and 66% of fatal traces are capability-limited: the model solves the problem from scratch at most one time in eight, so no step of the trace is a choke point. This is the counterfactual counterpart of Section 4's difficulty result, and it grows with the share of hard problems in a model's traces. Shares at $\delta \in \{1/16, 1/4\}$ move by a few points and do not change the ordering (released tables).

When the outcome is still open, the annotated error is often, but not usually exclusively, where it closes. Among fatal traces that are not capability-limited, the first error is the largest drop in v in 0.529 [0.353, 0.706], 0.344 [0.188, 0.531] and 0.423 [0.310, 0.535] of traces, against a position-matched null of 0.171, 0.214 and 0.159 (the Qwen2-1.5B

Table 3: Where fatal traces commit ($\delta = 1/8$; 95% bootstrap CIs in the text). "Decisive": share of non-capability-limited fatal traces whose largest drop in v is at the annotated first error; null: position-matched.

	Qwen2.5-1.5B	Qwen2-1.5B	Qwen2.5-Math-7B
Traces (clean/fatal/rec.)	53/51/12	15/67/6	60/207/47
Capability-limited	0.333	0.522	0.657
Latent commitment ($c < e$)	0.216	0.134	0.063
Commits at error ($c = e$)	0.373	0.239	0.179
Recoverable error ($c > e$)	0.078	0.104	0.101
Decisive = first error	0.529	0.344	0.423
position-matched null	0.171	0.214	0.159
n	34	32	71
Drop at error: fatal	+0.238	+0.140	+0.109
Drop at error: recovered	-0.099	+0.208	-0.215
Noise floor (clean)	+0.081	+0.208	+0.078

interval includes its null). Errors at the final step, where v_n is the observed outcome, are rare (0.056 for the 7B model) and excluding them leaves the shares unchanged (0.515, 0.296, 0.418). The rest of these traces split between *latent commitment*, where success has already collapsed before the written error (32%, 28% and 18% of non-capability-limited fatal traces), and *recoverable errors*, after which success stays above δ until a later step (12%, 22% and 30%).

Event study. Aligned at the annotated error (Fig. 3), fatal traces show a step change from $r = -1$ to $r = 0$ (0.270→0.032, 0.193→0.053, 0.175→0.066), while clean traces aligned at a random step drift smoothly upward, as the analogue of Eq. 1 for successes predicts. For recovered traces of the 7B model, success *rises* across the annotated error (0.374→0.589), by far more than the success-conditioned drift of clean traces (0.867→0.922): for this model, a step that humans mark as the first error often makes the correct answer more, not less, likely.

5.3 Where to backtrack

Verifier-guided search resamples a failing trace from some earlier point. We compare four targets on fatal traces: restarting from scratch (v_0), resampling from just before the annotated first error (v_{e-1}), resampling from just before the decisive step (v_{d-1}), and the best prefix in hindsight (an in-sample upper bound). The decisive step is an oracle quantity, but choosing it and scoring it on the same continuations would reward noise. We therefore split the m continuations of every prefix at random into halves A and B; half A decides whether a trace is not capability-limited ($v_0^A > \delta$) and where its decisive step is, and half B scores every policy. We average over 50 random splits and bootstrap over traces. On a planted-effect simulation in which the first error is the decisive step by construction, this estimator returns a difference of $+0.001$ between decisive-step and first-error backtracking, so it is not biased by the selection.

Backtracking to the annotated first error is worse than starting over. In all three models, resampling from just before the human-annotated first error succeeds less often than restarting from scratch, by +0.106 [+0.027, +0.187], +0.215 [+0.095, +0.339] and +0.068 [+0.007, +0.133] (Table 4, Fig. 4). These are traces where the model can solve the problem, and the target is the expert label, i.e. a perfect ProcessBench verifier. The correct steps before the error already lower the chance of success: the prefix carries the failure before the error is written.

Table 4: Success after resampling a fatal trace from each target (sample-split, 50 splits; mean pool size in parentheses). Differences are paired, with 95% trace-bootstrap CIs.

	Qwen2.5-1.5B (32)	Qwen2-1.5B (28)	Math-7B (76)
Restart	0.481	0.621	0.470
Before first error	0.375	0.406	0.402
Before decisive step	0.493	0.601	0.511
Best prefix (oracle)	0.574	0.688	0.622
Restart − error	+0.106 [+0.027, +0.187]	+0.215 [+0.095, +0.339]	+0.068 [+0.007, +0.133]
Decisive − error	+0.117 [+0.059, +0.183]	+0.196 [+0.082, +0.316]	+0.108 [+0.051, +0.165]
Decisive − restart	+0.011 [-0.035, +0.061]	-0.020 [-0.057, +0.020]	+0.040 [-0.008, +0.089]
Tokens: restart / decisive	374 / 308	203 / 173	650 / 463

Backtracking to where success drops recovers the loss at lower cost. Resampling from just before the decisive step beats first-error backtracking by +0.117, +0.196 and +0.108 (all CIs above zero; +0.055 for the 7B model with no selection at all) and matches restarting (+0.011, -0.020, +0.040; all CIs include zero) while generating 18%, 15% and 29% fewer continuation tokens. The best prefix in hindsight reaches 0.574, 0.688 and 0.622, so there is headroom for a target chosen better than either.

Reading. Eq. 1 explains why restart is a strong baseline: for any backtracking point at a fixed depth, or at a stopping time of the trace, prefixes of failed on-policy traces are on average no better than the empty prefix. The point just before the first error is neither (it depends on the next step), so the inequality does not force first-error backtracking below restart; the data show that it falls below anyway. A target helps only if it is chosen with information about where success is lost. Correctness labels do not provide that information; value drops do. Verifier-guided backtracking should therefore be evaluated against restart at matched compute, and verifiers meant to steer search should be trained to find drops in v, not first errors.

6 Discussion

For early warning. Reported AUROCs for early failure prediction should be accompanied by a question-only and a generator baseline, the increment over them, and a within-problem concordance; without them a detector of problem difficulty is indistinguishable from a detector of reasoning failure. On ProcessBench, text detectors pass this test only after half the chain.

For process supervision. Human first-error labels mark where reasoning first goes wrong, which is the right target for auditing reasoning. They are not, in general, where the outcome is decided, and as backtracking targets they are worse

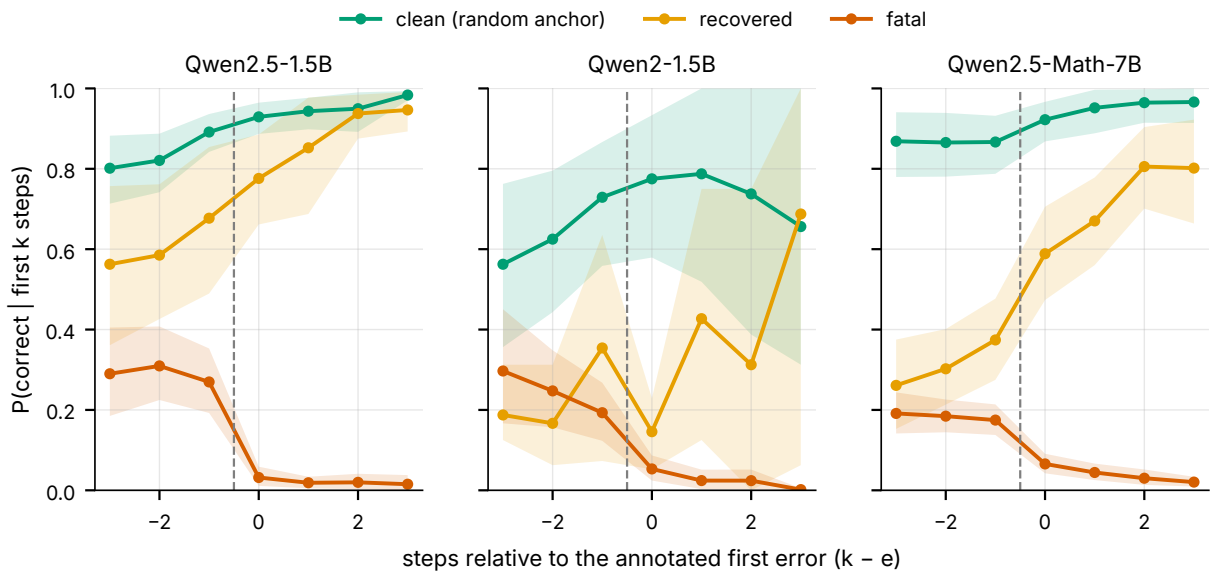


Figure 3: Success probability v_k aligned at the annotated first error ($k - e$; dashed line between the last correct step and the error). Shaded: 95% bootstrap CI. Clean traces are aligned at a uniformly drawn step (placebo).

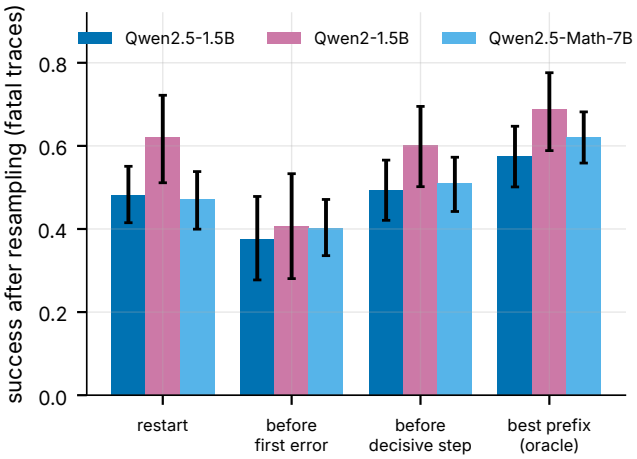


Figure 4: Success after resampling fatal traces from each target (half-B estimates, 50 splits).

than no target at all: a perfect ProcessBench verifier used to choose where to resample loses to restarting. Verifier-guided search should report a restart baseline at matched compute, and verifiers meant to steer search should be trained on drops in the counterfactual value curve, which labels exactly the step where success is lost.

What a trajectory can tell a detector. For the capability-limited failures (33–66%), the model fails at least seven times in eight before writing a single step, so reading the trajectory can add little beyond the problem and the model. This is the counterfactual counterpart of the difficulty-controlled result of Section 4, and it explains why text detectors gain only late in the chain: the failures they could still catch are the minority whose outcome is open, and those commit at a specific step.

7 Limitations

ProcessBench balances answers within each tier, so absolute fatality rates describe the benchmark, not deployment. The coded sample is small and was coded by one person; we release it for re-coding. Counterfactual continuations are sampled at temperature 0.7 from models whose original traces were re-segmented by ProcessBench, so prefixes are near- rather than exactly on-policy, and v_k describes the sampling distribution rather than the decode that produced the trace. Continuations of the 7B model are capped at 1,024 tokens, which counts truncated solutions as failures. The decisive step requires rollouts at every prefix and is an oracle target; we show where a verifier should point, not yet a cheap verifier that points there. Pools of non-capability-limited fatal traces are small (about 30 for each 1.5B model), and Qwen2-1.5B's clean traces are few, so its noise floor is high. We study mathematics only, and open models of at most 7B parameters.

8 Conclusion

Errors are where reasoning first goes wrong; choke points are where it stops being recoverable. On ProcessBench the two differ often enough, and text-based failure detectors confuse them with problem difficulty often enough, that both process supervision and early warning should measure the second directly. The counterfactual value curve does this at the cost of sampling. It shows that a third of failures or more are decided before the first step, that the annotated error is the choke point in about half of the rest, and that backtracking to it is worse than starting over, while backtracking to the drop in success matches a restart at lower cost. Its labels are the natural training target for verifiers that steer search.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS*, 2022.

[2] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," arXiv:2305.20050, 2023.

[3] J. Uesato *et al.*, "Solving math word problems with process- and outcome-based feedback," arXiv:2211.14275, 2022.

[4] P. Wang *et al.*, "Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations," in *ACL*, 2024.

[5] C. Zheng, Z. Zhang, B. Zhang, R. Lin, K. Lu, B. Yu, D. Liu, J. Zhou, and J. Lin, "ProcessBench: Identifying process errors in mathematical reasoning," in *ACL*, 2025.

[6] Z. Zhang *et al.*, "The lessons of developing process reward models in mathematical reasoning," arXiv:2501.07301, 2025.

[7] G. Tyen, H. Mansoor, V. Cărbune, P. Chen, and T. Mak, "LLMs cannot find reasoning errors, but can correct them given the error location," in *Findings of ACL*, 2024.

[8] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *ICLR*, 2024.

[9] A. Afzal, F. Matthes, G. Chechik, and Y. Ziser, "Knowing before saying: LLM representations encode information about chain-of-thought success before completion," arXiv:2505.24362, 2025.

[10] W. Lugoloobi, T. Foster, W. Bankes, and C. Russell, "LLMs encode their failures: Predicting success from pre-generation activations," arXiv:2602.09924, 2026.

[11] Y. U. Bulut, "It's the problem, not the path: Budget and difficulty confounds in LLM reasoning trajectories," arXiv:2609.03436, 2026.

[12] P. C. Bogdan, U. Macar, N. Nanda, and A. Conmy, "Thought anchors: Which LLM reasoning steps matter?" arXiv:2506.19143, 2025.

[13] Z. Lin *et al.*, "Critical tokens matter: Token-level contrastive estimation enhances LLM's reasoning capability," in *ICML*, 2025.

[14] E. Bigelow, A. Holtzman, H. Tanaka, and T. Ullman, "Forking paths in neural text generation," in *ICLR*, 2025.

[15] B. Liao *et al.*, "Lost at the beginning of reasoning," arXiv:2506.22058, 2025.

[16] S. Merrill and S. Srivastava, "The point of no return: Counterfactual localization of deceptive commitment in language-model reasoning," arXiv:2605.17113, 2026.

[17] D. Jiang, Z. Zhang, K. Yuan, and S. Ma, "Not every divergence should be suppressed: Counterfactual recoverability in on-policy distillation," arXiv:2608.04408, 2026.

[18] W. Li, F. Yang, A. Hazarika, S. A. Mehta, and K. Onoue, "When reasoning traces become performative: Step-level evidence that chain-of-thought is an imperfect oversight channel," arXiv:2605.11746, 2026.

[19] S. Boppana *et al.*, "Reasoning theater: Disentangling model beliefs from chain-of-thought," arXiv:2603.05488, 2026.

[20] M. Y. Cheung, A. Veeraraghavan, H. Chen, and G. Balakrishnan, "Conformal certification of reasoning trace prefixes," arXiv:2605.30085, 2026.

[21] H. Orgad *et al.*, "LLMs know more than they show: On the intrinsic representation of LLM hallucinations," arXiv:2410.02707, 2024.

[22] S. Kadavath *et al.*, "Language models (mostly) know what they know," arXiv:2207.05221, 2022.

[23] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *ICLR*, 2023.

[24] A. Yang *et al.*, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[25] A. Yang *et al.*, "Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement," arXiv:2409.12122, 2024.

[26] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *SOSP*, 2023.

[27] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *J. R. Stat. Soc. B*, vol. 69, no. 3, pp. 369–390, 2007.

[28] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.